

Lecture 10

Testing 2

Outline

- Code coverage
- White box testing techniques
 - Control flow testing technique
 - Data flow testing technique

References

- Daniel Galin, Software Quality Assurance: From theory to implementation, chapter 9
- Roger S. Pressman, Bruce R. Maxim, Software engineering: a practitioner's approach, part 3
- Boris Beizer, Software testing techniques

Code coverage

Code coverage

- **Coverage:** The ratio of components actually tested to the total components available.
- **Rule:** Higher coverage → Higher reliability
- **Several Criteria:**
 - **Statement coverage:** The line of code coverage granularity
 - **Decision/Branch coverage:** Control structure coverage granularity
 - **Condition coverage:** Boolean expression coverage granularity
 - **Paths coverage:** Every possible route coverage granularity

Code coverage

Statement coverage:

- Let A = The number of statements executed by test cases
- Let B = The total number of statements

$$\textit{Statement coverage} = \frac{A}{B}$$

Code coverage

Statement coverage:

- **Example:**

TC1: a = -1, b = 0 code coverage is 80%

TC2: a = 5, b = 0 code coverage is 100%

```
1 def foo(a, b):  
2     result = 0  
3     if a == 5 or b > 0:  
4         result = (10 / a) + (10 / b)  
5     return result
```

Code coverage

Statement coverage:

- **Capabilities:**
 - Unused statements
 - Dead code
 - Unused branches
 - Missing statements
- **Limitations:**
 - Complex logical expressions
 - Empty branches

Code coverage

Decision/Branch coverage:

- Let A = The number of decision outcomes (branches) executed by test cases
- Let B = The total number of decision outcomes (branches)

$$\textit{Decision coverage} = \frac{A}{B}$$

- **Goal:** Every decision in the program must take on all possible outcomes (true/false) at least once

Code coverage

Decision/Branch coverage:

- **Example:**

TC1: a = 5, b = 0 (True branch)

TC2: a = 1, b = 0 (False branch)

➤ 100% coverage with 2 TCs

```
1 def foo(a, b):  
2     result = 0  
3     if a == 5 or b > 0:  
4         result = (10 / a) + (10 / b)  
5     return result
```

Code coverage

Decision/Branch coverage:

- **Capabilities:**
 - Untested branches
 - Missing control flow
 - Guarantees statement coverage
- **Limitations:**
 - Hidden bugs in complex logical expressions

Code coverage

Condition coverage:

- Let A = The number of tested Boolean outcomes (conditions) executed by test cases
- Let B = The total number of Boolean outcomes (conditions)

$$\text{Condition coverage} = \frac{A}{B}$$

- **Goal:** Every condition in the program must take on all possible outcomes (true/false) at least once

Code coverage

Condition coverage:

- **Example:**

Every individual condition ($a == 5$ and $b > 0$) must take on all possible outcomes (T/F) at least once

$a == 5$: Tested as T (TC1) and F (TC2)

$b > 0$: Tested as F (TC2), but **never tested as T**

TC3: $a = 0$, $b = 1$

```
1 def foo(a, b):
2     result = 0
3     if a == 5 or b > 0:
4         result = (10 / a) + (10 / b)
5     return result
```

Code coverage

Condition coverage:

- **Capabilities:**
 - Condition-Level bugs
 - High granularity
- **Limitations:**
 - Does not guarantee decision coverage
 - Combinational explosion

White box testing techniques

White box testing techniques

- **White Box Testing** (also called structural or glass-box techniques) is based on knowledge of the design, internal logic and code structure
- **Goal:**
 - **Correctness:** Verify calculations and paths
 - **Quality:** Ensure standard compliance
 - **Maintenance:** Check for code maintainability and reusability
- **Key Strategies:**
 - Control flow testing (Basis path testing, Loop testing)
 - Data flow testing

Control flow testing technique

- **Control flow testing:** is a structural testing strategy that uses the program's control flow as a model
- **Basic Idea:** Select specific execution paths and generate inputs (test cases) to test them
- **Process:**
 - **Step 1:** Create a Control Flow Graph (CFG) from the design, either manually or automatically
 - **Step 2:** Define a coverage target over the CFG
 - **Step 3:** Test cases are created using CFG to cover the coverage target
 - **Step 4:** Execute the test cases
 - **Step 5:** Analyze the results and determine whether the program is error free or has some bugs

Basis path testing

- A technique first proposed by Tom McCabe to measure program complexity and derive test cases.
- Uses logical complexity to define a “basis set” of execution paths
- **Guarantee:** Executes every statement in the program at least once
- **Process to Generate the Basis Set:**
 - **Draw:** Create a flow graph from the design
 - **Calculate:** Determine the flow graph's cyclomatic complexity
 - **Identify:** Find the basis set of linearly independent paths
 - **Test:** Prepare test cases to force execution of each path

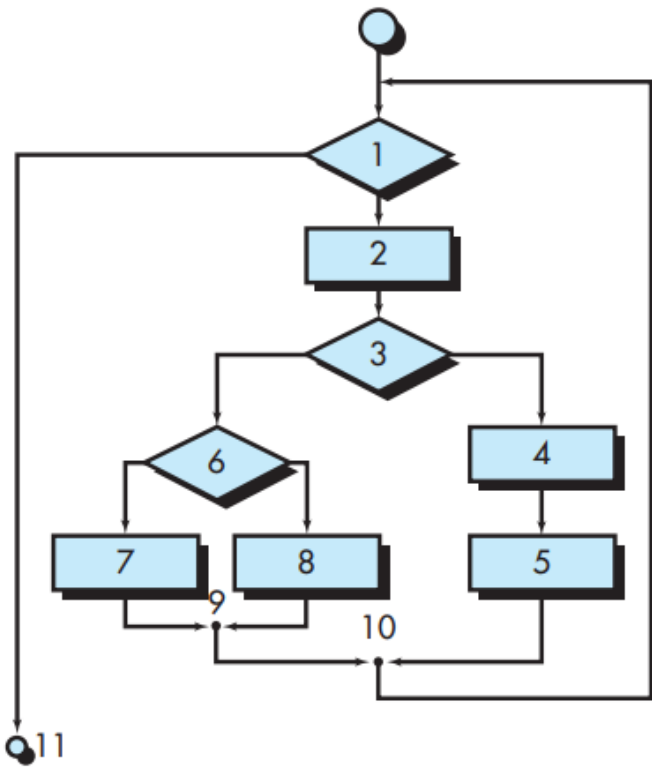
Basis path testing

Control Flow Graph (CFG)

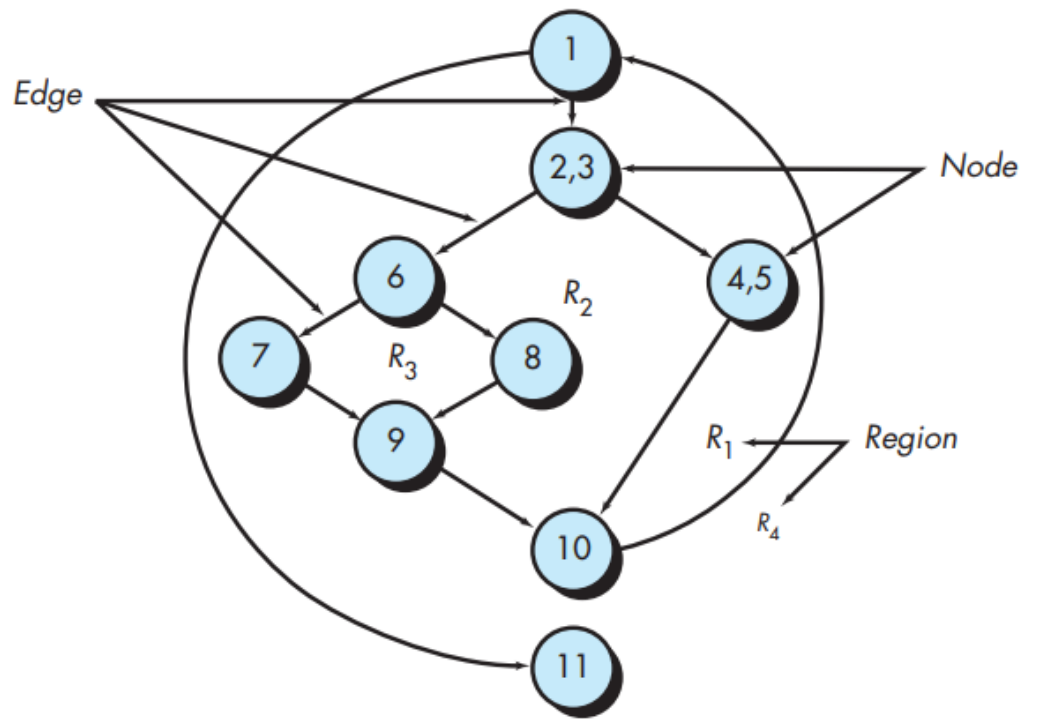
- A graphical representation of the control structure of a program
- **Key Notations:**
 - **Node:** Represents one or more procedural statements
 - **Edges or links:** Represent the flow of control within the program
 - **Decision node:** A node that has more than one arrow leaving it
 - **Junction node:** A node that has more than one arrow entering it
 - **Regions:** Areas that are bounded by edges and nodes

Basis path testing

Control Flow Graph (CFG)



(a)



(b)

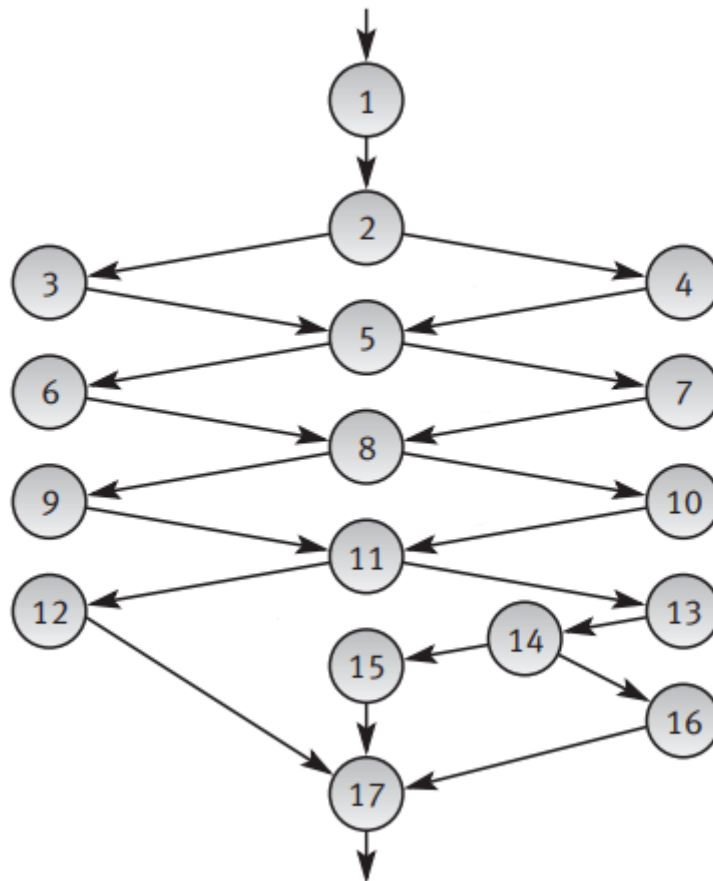
Basis path testing

Cyclomatic Complexity

- Measure the complexity of a module and determine the maximum number of independent paths to be tested
- Cyclomatic complexity is computed using CFG
- **Formula:**
 - $V(G) = E - N + 2$
where E: Number of edges, N: Number of nodes
 - $V(G) = P + 1$
where P: Number of decision nodes
 - $V(G) = R$
where R: Number of regions in the flow graph

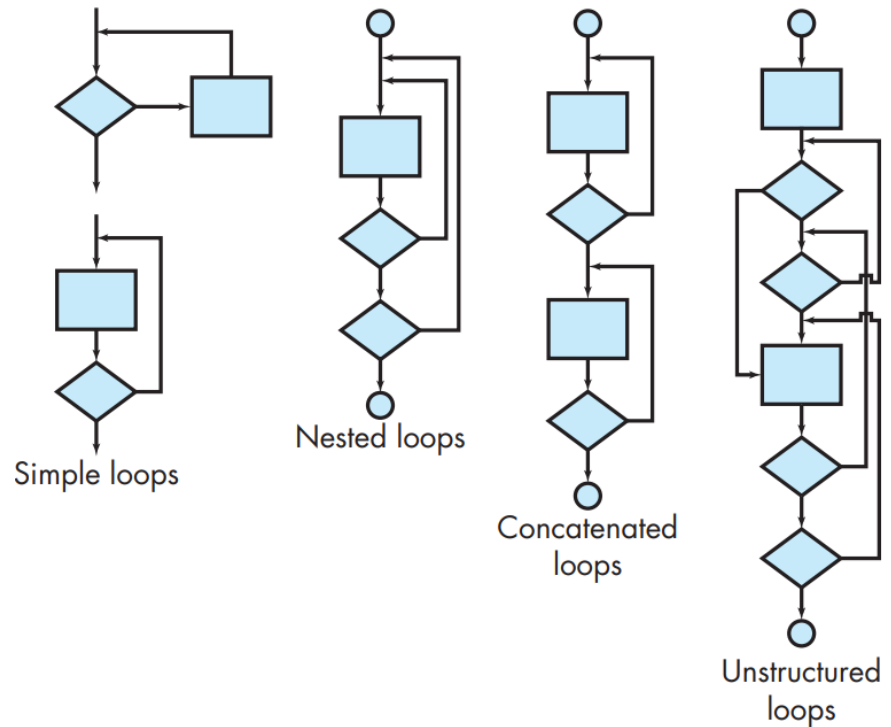
Basis path testing

Cyclomatic Complexity



Loop testing

- A technique focuses exclusively on checking the validity of the loop structure
- Four different classes of loop: simple loops, concatenated loops, nested loops, and unstructured loops



Loop testing

For simple loops:

With n is maximum number of iterations

- Skip the loop entirely
- Exactly 1 time
- Exactly 2 times
- Loop m times ($k < n$)
- Loop $n-1, n$, and $n+1$ times

Loop testing

For nested loops:

To avoid an impractical, geometric growth of test cases, follow this inside-out approach:

- **Start innermost:** Set all outer loops to their minimum values
- Conduct simple loop tests on the innermost loop
- **Work outward:** Test the next outer loop while keeping other outer loops at minimums and nested loops at “typical” values
- Continue working outward until all loops are tested

Loop testing

For concatenated loops:

- **If independent:** Treat and test them exactly like Simple loops
- **If dependent** (e.g., loop 1's counter is used as loop 2's initial value): Treat and test them using the Nested loops approach

For unstructured loops:

- Whenever possible, these class loops should be redesigned

Loop testing

Example:

```
1 def process_items(num_items):  
2     n = 5  
3     count = 0  
4     while count < num_items and count < n:  
5         count += 1  
6     return count
```

Input	Expected Loop Passes	Output (count)
0	0	0
1	1	1
2	2	2
3	3	3
4	4	4
5	5	5
6	5	5

Data flow testing technique

- A white-box testing technique focused on:
 - Statements where variables are assigned values
 - Statements where these values are referenced
- Bugs Detected:
 - **Initialization:** Missing definitions before use
 - **Assignment:** Incorrect or redundant data entry
 - **Sequence:** Data manipulated in the wrong logical order
- Life cycle principle: Every variable must follow a strict 3-step sequence: **Created**→**Used**→**Deleted**

Data flow testing technique

Variable life cycle analysis:

- **3 Main Actions on a Variable:**
 - **d (define):** Assigns a value or inputs data
 - **r (reference):** Uses the variable's value
 - **u (undefine):** Deletes or cancels the variable
- **Initial state (Symbol ~):** The variable does not exist yet
 - ~d: Defined correctly (Normal)
 - ~r: Used before being defined → **Error** (No value exists)
 - ~u: Deleted before being defined → **Abnormal**

Data flow testing technique

9 Pairs of Variable Operations:

- **Valid & Normal sequences:**
 - dr (Define $\rightarrow$ Reference): Correct and normal sequence
 - rd (Reference $\rightarrow$ Define new value): Reasonable
 - rr (Reference $\rightarrow$ Reference): Reasonable
 - ru (Reference $\rightarrow$ Undefine): Reasonable
 - ud (Undefine $\rightarrow$ Redefine): Acceptable

Data flow testing technique

9 Pairs of Variable Operations:

- **Suspicious sequences (Potential bugs):**
 - dd (Define $\rightarrow$ Define again): A bit strange, possibly a programming error (useless overwrite)
 - du (Define $\rightarrow$ Undefine): A bit strange, deleted before being used
- **Definite Error sequences:**
 - ur (Undefine $\rightarrow$ Reference): **SEVERE BUG**
 - uu (Undefine $\rightarrow$ Undefine again): Highly likely a programming error

Data flow testing technique

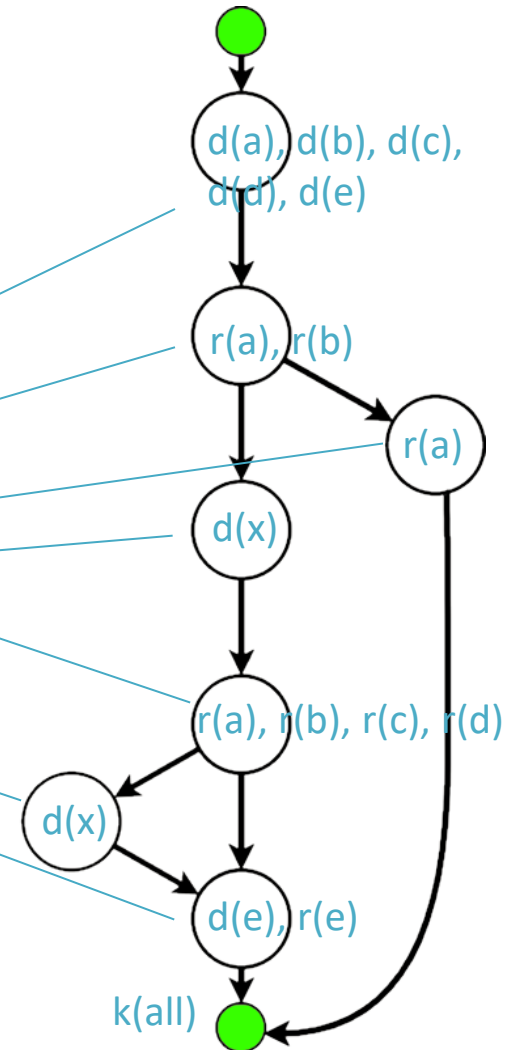
- **Process:**
 - **Step 1:** Create a Data Flow Graph (DFG) from the CFG by labeling variable actions (e.g., $d(x)$, $u(a)$) on the nodes/edges. The purpose is to describe different life cycle scenarios of the variable
 - **Step 2:** Calculate Cyclomatic Complexity $V(G)$
 - **Step 3:** Create paths based on $V(G)$
 - **Step 4:** Test Variable Life Cycle:
 - Each variable will have a maximum of $V(G)$ scenarios corresponding to the paths
 - Iterate through each scenario to find **abnormal action pairs**

Data flow testing technique

Example:

- Graph has 2 decision node, then
 $V(G) = 2 + 1 = 3$
- Function has 4 input parameter and 2 local variables: a, b, c, d, e, x

```
1 def foo(int a, int b, int c, int d):  
2     int e  
3     if (a > b):  
4         return a  
5     int x = 0  
6     if ((a == b) or (c == d) and (a > 0)):  
7         x = 1  
8     e = 1/x  
9     return e
```

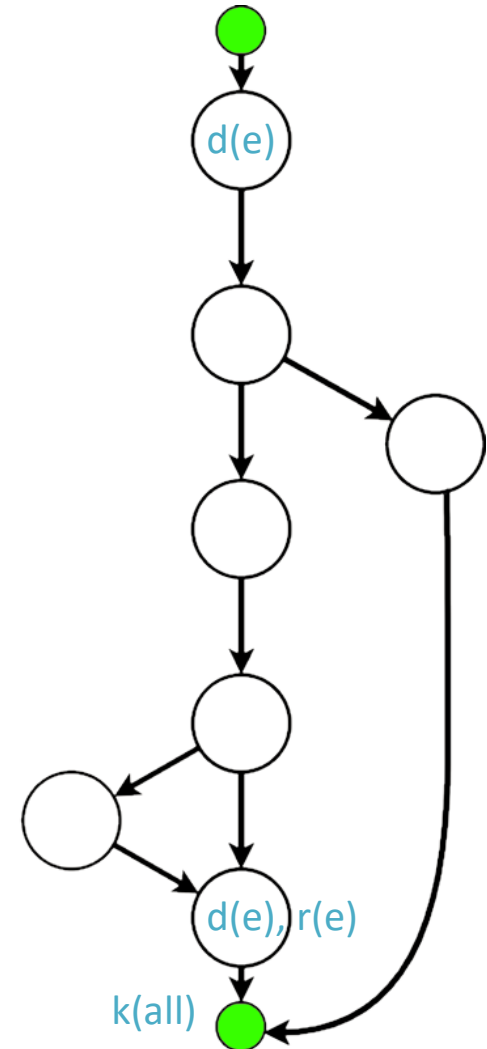


Data flow testing technique

Example:

- Scenario 1: $\sim dduk$
- Scenario 2: $\sim dduk$ (like scenario 1)
- Scenario 3: $\sim dk$

Scenario 1 & 2 have abnormal pair dd
So, we must determine whether it is
related to an error or not



Advantages of White box testing

- **Direct Code Checking:** Verifies processing paths and algorithms line-by-line for absolute correctness
- **Line Coverage Follow-up:** Identifies unexecuted lines of code, allowing testers to create specific test cases to cover them
- **Coding Quality:** Ensures the code strictly adheres to required coding standards

Disadvantages of White box testing

- **Vast Resources Utilized:** Requires significantly more time, effort, and money compared to black box testing
- **Inability to Test Performance:** Cannot evaluate non-functional factors:
 - Availability (Response time)
 - Reliability
 - Load durability
- **Limited Application:** Only practical for software modules with very high risk and very high cost of failure

Summary

- Code coverage
- White box testing techniques
 - Control flow testing technique
 - Data flow testing technique